Anno Accademico 2026/2027

Leonardo Donati

Appunti

Sistemi Operativi Avanzati

Teoria e esercizi.



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Indice

1 Introduzione **3**
1.1 Sistema Operativo . . . 3
1.2 Organizzazione del Sistema di un Computer . . . 4
1.2.1 Interrupt . . . 4
1.2.2 Struttura della Memoria . . . 6
1.3 Architetture dei Computer . . . 6
1.4 Operazioni di un Sistema Operativo . . . 6
1.4.1 Multitasking e Multiprogramming . . . 7
1.4.2 Operazioni Dual-Mode . . . 7
1.4.3 Timer . . . 7
1.5 Sicurezza e Protezione . . . 7

2 Chiamate di Sistema **9**
2.1 Application Programming Interface (API) . . . 9
2.1.1 Tipi di Chiamate di Sistema . . . 10
2.2 Servizi di Sistema . . . 11

Capitolo 1

Introduzione

1.1 Sistema Operativo

> **DEFINITION** Sistema Operatvio
>
> Un sistema operatvio è un **software** che agisce come intermediario fra l'hardware e l'utente.

Ha diversi obiettivi:

- Eseguire programmi dell'utente.
- Far diventare l'uso del dispositivo conveniente.
- Usare l'hardware del computer in modo efficiente.

Per capire bene cosa fa un sistema opertivo è utili dividere la struttura di un computer in quattro parti principali:

1. **Hardware:** provvede le risorse basi, come la cpu, la memoria centrale e i dispositivi di I/O.
2. **Sistema operativo:** controlla e gestisce l'hardware per applicazioni e utenti.
3. **Programmi applicativi:** decide come le risorse hardware devono essere divise fra i vari programmi.
4. **Utenti:** agenti esterni che interagiscono con il computer.

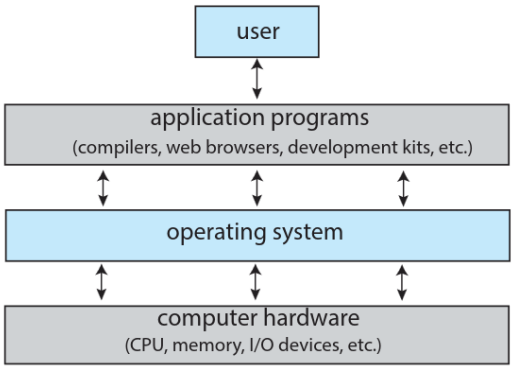


Figura 1.1: Architettura del sistema di un computer.

Dal punto di vista di un utente, deve essere facile usare applicazioni e avere buone prestazioni.
Dal punto di vista del computer, però, tutti gli utente devono essere soddisfatti, quindi il sistema operativo deve essere un **allocatore di risorse** e un **gestore di programmi**.

1.2 Organizzazione del Sistema di un Computer

Un computer moderno **general-purpose** consiste in una o più CPU e un numero di dispositivi che sono collegati tramite un **bus** comune che da accesso alla memoria condivisa e agli altri componenti.

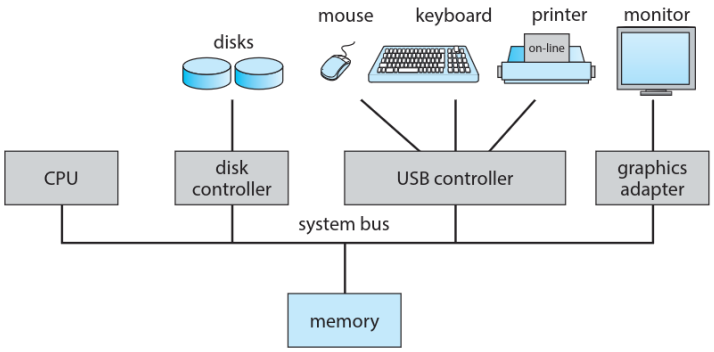


Figura 1.2: Organizzazione del sistema di un computer.

Ogni dispositivo ha un controller, che si occupa di gestire i dati fra il suo buffer interno e altri componenti. I sistemi operatvi per interagire in modo uniforme con i controller, usano dei **driver**. Dato che la CPU e i controller possono agire in parallelo, competendo per cicli di memeoria, bisogna trovare un modo ordinato per l'accesso alla memoria condivisa.

1.2.1 Interrupt

EXAMPLE Operazione di I/O

Per un'operazione di I/O si succedono diverse azioni:

- Il driver carica correttamente i registri del controller.
- Il controller guarda i regsitri per capire che operazione fare.
- Il controller carica i dati da un dispositivo al suo buffer locale.
- Il controller informa il driver di aver finito l'operazione.

Per ultimo, il driver da il controllo al sistema operativo, passando i dati letti o un puntatore a questi.

Il controller per avvisare il driver di aver finito la sua operazione, usa una **interrupt**.

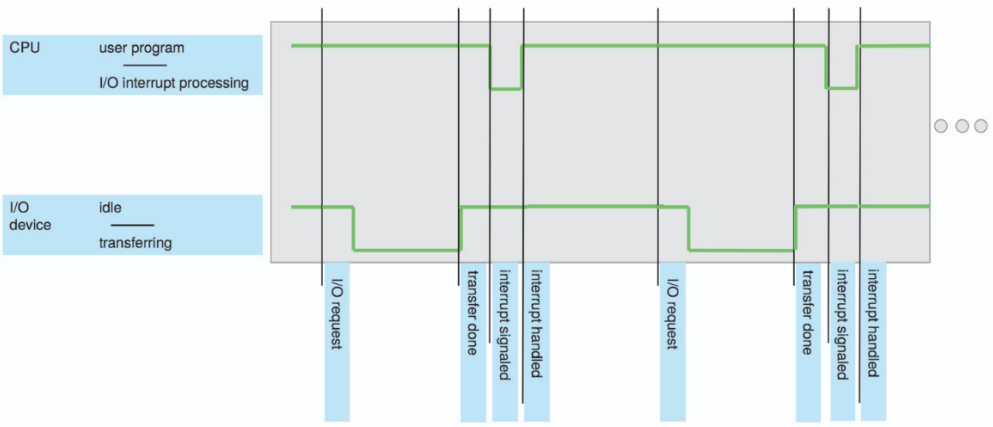


Figura 1.3: Timeline di una interrupt.

Quando la CPU riceve una interrupt, ferma immediatamente quello che sta facendo e trasferisce l'esecuzione alla service routine corretta, fino a che non è stata completata. Una volta finita, la CPU ricomincia da dove aveva interrotto, inserendo nel **program counter** l'indirizzo corretto.

> **NOTA BENE**
>
> Questa metodologia richiede, però, di dover salvare lo stato delle informazioni della CPU, dato che le service routine degli interrupt potrebbero modificare i registri.

Le interrupt, dato che sono molto frequenti, devono essere gestite velocemente. Per questo, si può usare una tabella, salvata nella parte basse della memoria, che contiene tutti gli indirizzi delle service routine degli interrupt, garantendo la velocità necessaria. Questo array (o **vettore degli interrupt**) di indirizzi, ha come indici dei numeri univoci che specificano la routine corretta per il tipo di interrupt.

vector number	description
0	divide error
1	debug exception
2	null interrupt
3	breakpoint
4	INTO-detected overflow
5	bound range exception
6	invalid opcode
7	device not available
8	double fault
9	coprocessor segment overrun (reserved)
10	invalid task state segment
11	segment not present
12	stack fault
13	general protection
14	page fault
15	(Intel reserved, do not use)
16	floating-point error
17	alignment check
18	machine check
19–31	(Intel reserved, do not use)
32–255	maskable interrupts

Figura 1.4: Tabella degli indici di un vettore di interrupt.

Implementazione

L'hardware della CPU ha un cavo chiamato **interrupt-request line**, dove la CPU riceve i segnali dai controller. Una volta intercettato il numero dell'interruzione, tramite il vettore degli interrupt, viene chiamata l'**interrupt-handler routine** corretta.
In sistemi operativi moderni, il metodo deve essere più sofisticato:

- È necessaria la possibilità di differire la gestione delle interruzioni durante l'esecuzione di elaborazioni critiche.
- È necessario un metodo efficiente per indirizzare l'esecuzione all'interrupt-handler corretto per un determinato dispositivo.
- Sono necessarie interruzioni multilivello, in modo che il sistema operativo possa distinguere tra interruzioni ad alta e bassa priorità e rispondere con il livello di urgenza adeguato.

Queste tre caratteristiche sono fornite dalla CPU e dall'**interrupt-controller hardware**.

Per gestire la priorità tra interrupt, nelle CPU esistono due linee di richieste per le interruzioni:

1. **Maskable:** può essere momentaneamente spenta dalla CPU, per dare priorità a istruzioni critiche. Questa linea è usata dai controller dei dispotivi.
2. **Non-Maskable:** usata per interrupt che non possono essere ignorate, come errori di memoria non recuperabili.

1.2.2 Struttura della Memoria

La memoria di un computer è organizzata in un gerarchia che tiene conto di tre valori:

1. **Velocità**.
2. **Capacità**.
3. **Volatilità**.

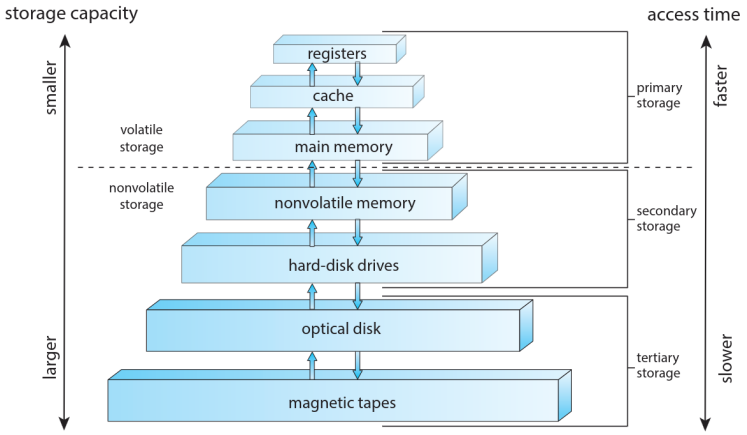


Figura 1.5: Gerarchia delle memorie.

Per spostare grandi quantità di dati, la metodologia degli interrupt descritta prima potrebbe creare molto overhead, per questo viene usata la **DMA** (**Direct Memory Access**).

> **DEFINITION** Direct Memory Access
>
> Dove aver sistemato buffer e puntatori, il controller del dispositivo sposta un grande blocco di dati dalla memoria al componente, o viceversa, senza l'intervento della CPU. Questo genera una sola interrupt per blocco, a discapito delle numerose interrupt generate senza DMA.

1.3 Architetture dei Computer

Possono esistere diverse architetture, basate sul numero di processori presenti all'interno del computer:

- **Single-Processor:** se è presente un solo processore.
- **Multi-Processor:** attualmente comune come **Multi-Core**, se sono presenti più processori. Condividono la memoria centrale e i dispositivi di I/O.
- **Cluster:** si distinguono dai computer multi-processor, perchè sono sistemi autonomi che lavorano insieme. Ogni nodo è spesso un computer multi-core.

> **DEFINITION** Alcune concetti di base
>
> **CPU:** hardware che esegue le istruzioni.
> **Processor:** chip che contiene una o più CPU.
> **Core:** unità basica della CPU.

1.4 Operazioni di un Sistema Operativo

Per avviare il computer, è necessario un programma di iniziale: il **bootstrap**. Serve a inizializzare tutte le risorse utili, come registri e i vari controller. In ultimo, deve localizzare il kernel del sistema operativo e caricarlo in memoria.

Il sistema operativo, in assenza di azione da eseguire, aspetta degli eventi, come le interrupt a livello hardware o le **trap** (**exceptions**) a livello software.

1.4.1 Multitasking e Multiprogramming

L'obiettivio del sistema operativo è tenere la CPU occupata tutto il tempo e nel mentre garantire all'utente di utilizzare più programmi contemporaneamente. Tutto questo è fattibile grazie alla capacità di essere multiprogramming, potendo gestire più programmi in esecuzione o **processi**.
Invece, il multitasking, è la capacità di poter saltare da un processo all'altro velocemente, garantendo un tempo di risposta molto basso all'utente.

1.4.2 Operazioni Dual-Mode

PROBLEMA

Il sistema operativo e gli utenti risiedono sullo stesso hardware. Si deve trovare una soluzione per evitare di eseguire codice malevolo che potrebbe alterare il funzionamento di altri programmi o del sistema stesso.

Come minimo, sono necessarie due modalità: **Kernel Mode** e **User Mode**. All'hardware è stato aggiunto un bit, chiamato **bit mode**, che permette di distinguere la modalità corrente: kernel (0) e user (1).

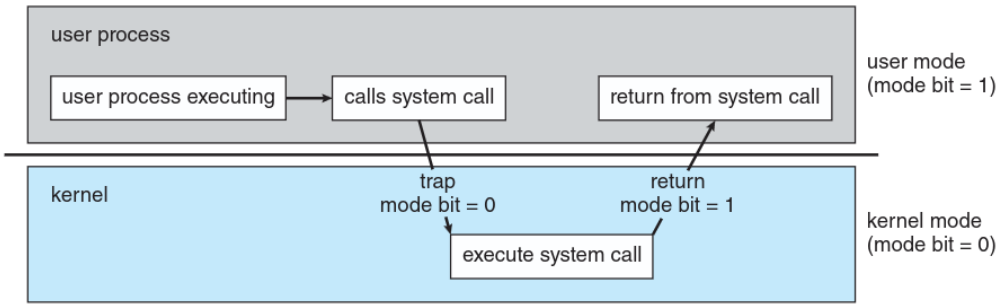


Figura 1.6: Operazione Dual-Mode.

Se si prova a eseguire una istruzione vietata in user mode, allora l'hardware lancerà una trap al sistema operativo, il quale tramite il vettore delle interrupt, prende il controllo e termina l'esecuzione del processo.

1.4.3 Timer

Bisogna assicurarsi che il sistema operativo mantenga sempre il controllo sulla CPU, per evitare che programmi in esecuzione dell'utente si blocchino in un loop infinito, oppure che una system call fallita non ritorni il controllo al sistema operativo.
Una soluzione è usare un **timer**, che interrompe il computer dopo un certo periodo. Solitamente, viene

implementato tramite un clock a frequenza fissa e un contatore. Ogni ciclo di clock, il contatore viene diminuito di una unità.

TIMER SU SISTEMI LINUX

Nei sistemi Linux, il parametro di configurazione del kernel `HZ` specifica la frequenza delle interrupt del timer. Per esempio, `HZ` = 250, significa 250 interruzioni al secondo o una ogni 4 millisecondi.

1.5 Sicurezza e Protezione

Nei sistemi operativi moderni, che supportano più utenti, i dati che vengono usati devono essere garantita solo al processo che ha ricevuto tutte le autorizzazioni.

DEFINITION Protezione

Qualsiasi meccanismo per controllare l'accesso alle risorse di un computer da parte di utenti o processi. Deve essere possibile avere un modo per specificare i controlli e controllare che vengano rispettati.

Anche se è presente un alto grado di protezione, può rimanere comunque vulnerabile a fallimenti o accessi non autorizzati.

> **DEFINITION** Sicurezza
>
> Difendere il sistema da attacchi interni ed esterni, come dos, worms, furti di dati e identità, ecc...

I sistemi operativi per distinguere fra gli utenti, usa diversi identificativi:

- **User ID:** nome e numero associato a ogni utente.
- **Group Identifier:** per definire un gruppo di utenti e associargli risorse e processi.

Per ottenere più permessi, è possibile fare **privilage escalation** per cambiare il proprio ID e avere più permessi.

Capitolo 2

Chiamate di Sistema

DEFINITION Chiamata di Sistema

Interfaccia ai servizi principali, messa a disposizione da il sistema operativo. Solitamente, queste chiamate sono funzioni scritte in C/C++ o per alcuni casi specifici in linguaggio Assembly.

2.1 Application Programming Interface (API)

Nei sistemi moderni, vengono eseguite migliaia di system calls ogni secondo, quindi, gli sviluppatori, per realizzare le applicazioni si affidano a una **Application Programming Interface** (**API**).

DEFINITION API

Insieme di funzioni disponibili per un programmatore di applicazioni, inclusi i parametri passati a ciascuna funzione e i valori di ritorno che il programmatore può aspettarsi.

Le API più comuni sono quella di Windows, POSIX (Unix, Linux e MacOS) e quella per la Java Virtual Machine.

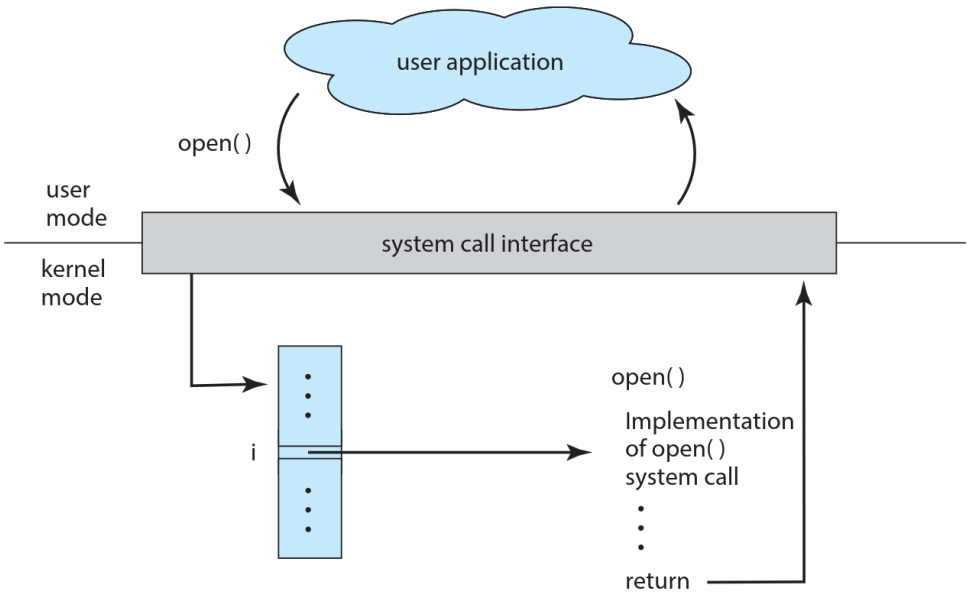


Figura 2.1: Esempio di System Call.

Un fattore nella gestione delle chiamate di sistema è l'**ambiente di esecuzione** (**RTE**), ossia l'insieme di software necessari per eseguire applicazioni scritte in un determinato linguaggio di programmazione, inclusi i relativi compilatori o interpreti, oltre ad altri componenti come librerie e loader.

L'RTE, tramite una **system call interface**, fa da link fra le system call al sistema operativo e le istruzione dell'API.
Tipicamente, a ogni chiamata di sistema viene associato un numero e viene mantenuta una tabella indicizzati secondi questi valori per gestire le call.

I parametri delle system call, vengono passati tramite registri, nel caso non sia possibile, tramite blocchi memoria oppure con una push sullo stack, dove poi il sistema oprativo eseguirà una pop.

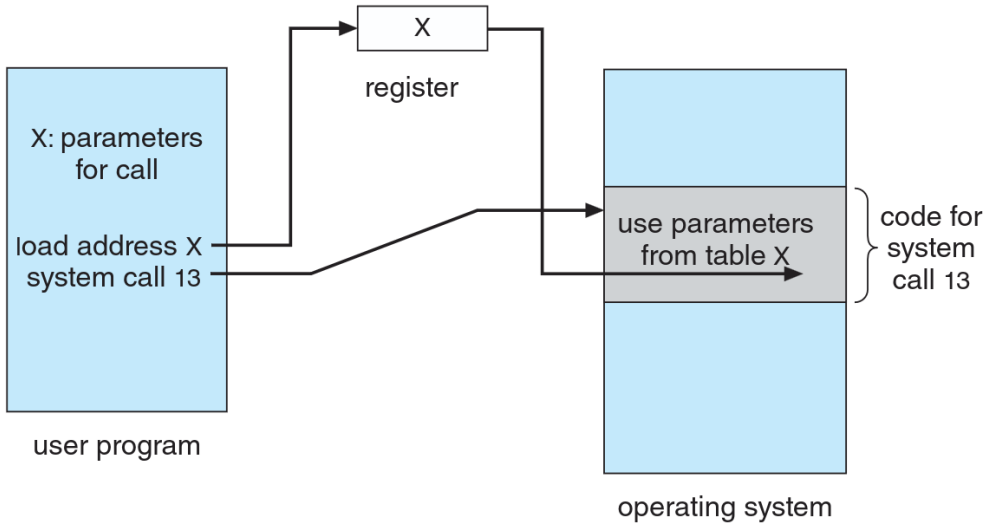


Figura 2.2: Esempio di Passaggio di Parametri.

2.1.1 Tipi di Chiamate di Sistema

Esistono sei tipi di system calls:

1. **Process Control**:
   (a) Creazione e terminazione di processi;
   (b) Caricamento ed esecuzione;
   (c) Eventi e segnali di wait;
   (d) Allocazione e liberazione della memoria;
   (e) Get e set di attributi dei processi.
2. **File Management**
   (a) Creazione ed eliminazione di file;
   (b) Apertura e chiusura di file;
   (c) Lettura e scrittura di file;
   (d) Get e set di attributi dei file.
3. **Device Management**:
   (a) Richiesta e rilascio un dispositivo;
   (b) Lettura e scrittura;
   (c) Get e set degli attributi di un dispositivi;
   (d) Attach e detach logico di dispositivi.
4. **Information Maintenance**:
   (a) Get e set di tempo e data;
   (b) Get e set di dati di sistema;
   (c) Get e set di attributi di file, processi e dispositivi.

5. **Communications**:
   (a) Creazione ed eliminazione canali di comunicazione;
   (b) Invio e ricezione di messaggi;
   (c) Trasferimento di informazioni di stato;
   (d) Attach e detach di dispositivi remoti.
6. **Protection**:
   (a) Get file permission;
   (b) Set file permission.

2.2 Servizi di Sistema

> **DEFINITION** Servizi di Sistema
>
> Offrono un ambiente conveniente per programmare ed eseguire applicazioni.
> Alcuni sono solamente warp di chiamate di sistema, altre invece sono più complessi.

Ci sono diverse categorie:

- **Gestione dei file:** permettono di eseguire diverse operazioni su file e directory.
- **Informazioni di Status:** permettono di interrogare l'SO per avere informazioni come data, tempo, spazio su memoria o dischi, ecc...
  Inoltre, permettono azioni più complesse come il logging, il debugging e l'analisi di prestazioni.
- **Modifica dei File:** modificare o creare file su dischi, tramite, per esempio, svariati text editor.
- **Supporto a linguaggi di Programmazione:** Compilatori, debugger e interpreti per linguaggi di programmazione tipo C/C++, Java, Python, ecc...
- **Esecuzione e Caricamento dei Programmi:** i programmi devono essere caricati in memoria ed eseguiti. Il sistema deve fornire linker e loader, oltre a i vari debugger.
- **Comunicazioni:** permettono di creare canali di comunicazioni virtuali fra processi, utenti o computer.
- **Servizi di Bakcground:** alcuni servizi vengono lanciati nel tempo di boot, alcuni terminano, altri invece continuano a lavorare finchè il sistema non viene spento.
  Normalmente, quest'utlimi sono chiamati **demoni**.